

Deploy a Slack Marketplace App on DigitalOcean

This guide shows how to deploy a Slack app for the Slack Marketplace/App Directory on a DigitalOcean Droplet using a public HTTPS endpoint.

This is different from the Socket Mode-only setup because a public Slack app must use HTTP request URLs and a publicly reachable HTTPS endpoint.

Why this guide is different

Slack Marketplace submission requires:

- a public HTTPS endpoint
- verified Request URLs for Event Subscriptions and Interactivity
- public distribution enabled in the Slack app configuration

For this reason, the deployment must expose a web endpoint that Slack can call over HTTPS.

1. Create the Droplet

Create a new Droplet with:

- OS: Debian 12 (or the latest Debian release supported by your tooling)
- Plan: at least 2 GB RAM if you expect regular traffic
- SSH keys enabled
- Firewall rules configured after initial setup

You can follow the DigitalOcean Debian setup guide for initial server setup.

2. Secure the server and configure firewall

Update the system and install the firewall:

```
sudo apt update
sudo apt install ufw
sudo ufw app list
sudo ufw allow OpenSSH
sudo ufw enable
sudo ufw status
```

Allow web traffic for your public Slack endpoint:

```
sudo ufw allow 'Nginx Full'
```

3. Install required tools

Install Docker, Git, Python, Node.js, Nginx, and Certbot:

```
sudo apt-get update
sudo apt-get install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/debian/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

echo \
  "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/debian \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update

sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-
plugin docker-compose-plugin
```

Test Docker:

```
sudo docker run hello-world
```

Install Python and build tools:

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip git curl build-
essential nginx certbot python3-certbot-nginx
python3 --version
pip3 --version
```

Install Node.js and npm:

```
sudo apt update
sudo apt upgrade -y
sudo apt install nodejs npm -y
```

4. Clone the repository

```
cd /home/$USER
git clone https://github.com/pinionai/pinionai-streamlit-agent.git
```

```
cd pinionai-streamlit-agent
```

If you want to use your own fork, replace the repository URL with your fork.

5. Create the Python environment

```
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

If your marketplace app uses a web framework such as Flask or FastAPI, install the additional dependencies required by your endpoint service.

6. Create the environment file

Create a `.env` file with your Slack and PinionAI credentials:

```
cp .env.example .env 2>/dev/null || touch .env
chmod 600 .env
nano .env
```

Example:

```
host_url=https://api.pinionai.com
client_id=<YOUR_CLIENT_ID>
client_secret=<YOUR_CLIENT_SECRET>
agent_id=<YOUR_AGENT_ID>

SLACK_BOT_TOKEN=xoxb-...
SLACK_SIGNING_SECRET=your-signing-secret
SLACK_CLIENT_ID=your-client-id
SLACK_CLIENT_SECRET=your-client-secret
```

Required values

- `SLACK_BOT_TOKEN`: bot token for sending messages
- `SLACK_SIGNING_SECRET`: verifies requests from Slack
- `SLACK_CLIENT_ID` / `SLACK_CLIENT_SECRET`: used for OAuth and installation flow
- `client_id` / `client_secret` / `agent_id`: your PinionAI agent settings

Keep `.env` private and do not commit it to Git.

7. Configure the Slack app for Marketplace submission

In the Slack API dashboard, configure the app as follows.

A. Enable public distribution

1. Open Settings → App Distribution.
2. Turn on public distribution.
3. Fill in the app profile details, support URL, privacy policy URL, and app icon.

B. Configure public HTTPS Request URLs

Slack expects public HTTPS endpoints for:

- Event Subscriptions
- Interactivity & Shortcuts
- Slash Commands

Use your public domain, for example:

- `https://slack.yourdomain.com/slack/events`
- `https://slack.yourdomain.com/slack/interactive`
- `https://slack.yourdomain.com/slack/commands`

C. Enable Event Subscriptions

1. Go to Features → Event Subscriptions.
2. Turn on Event Subscriptions.
3. Subscribe to the events your bot handles, such as:
 - `message.channels`
 - `message.groups`
 - `message.im`
4. Save and verify the Request URL.

D. Configure bot scopes

In Features → OAuth & Permissions, add the scopes your app requires, such as:

- `chat:write`
- `files:read`
- `channels:history`
- `groups:history`
- `im:history`

Install the app to your workspace and verify the OAuth flow.

8. Install and configure Nginx

Install Nginx:

```
sudo apt update
sudo apt install nginx
sudo ufw app list
sudo ufw allow 'Nginx Full'
```

Test that Nginx is running:

```
sudo systemctl status nginx
```

Create a site config:

```
sudo touch /etc/nginx/sites-available/slack.yourdomain.com
sudo ln -s /etc/nginx/sites-available/slack.yourdomain.com
/etc/nginx/sites-enabled/slack.yourdomain.com
sudo nano /etc/nginx/sites-available/slack.yourdomain.com
```

Example configuration:

```
server {
    listen 80;
    listen [::]:80;

    server_name slack.yourdomain.com;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_redirect off;
        proxy_set_header Host $http_host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 900;
    }
}
```

Test syntax and restart Nginx:

```
sudo nginx -t
sudo systemctl restart nginx
```

9. Secure Nginx with HTTPS using Certbot

Install Certbot and configure HTTPS:

```
sudo apt update
sudo apt install python3-acme python3-certbot python3-mock python3-openssl
python3-pkg-resources python3-pyparsing python3-zope.interface
sudo apt install python3-certbot-nginx
sudo certbot --nginx -d slack.yourdomain.com
```

Certbot will update the Nginx config automatically and provide the HTTPS endpoint Slack needs.

10. Run the Slack endpoint service

Your Slack Marketplace app should run behind the Nginx reverse proxy on a local port such as **8000**.

Example with **uvicorn**:

```
source .venv/bin/activate
uvicorn app:app --host 127.0.0.1 --port 8000
```

If you prefer **gunicorn**:

```
source .venv/bin/activate
gunicorn -w 2 -k uvicorn.workers.UvicornWorker app:app --bind
127.0.0.1:8000
```

You can verify the endpoint with:

```
curl https://slack.yourdomain.com/slack/health
```

11. Run the service as a systemd process

To keep the app running after reboots, create a systemd unit:

```
sudo nano /etc/systemd/system/pinionai-slack-marketplace.service
```

Example content:

```
[Unit]
Description=PinionAI Slack Marketplace App
After=network.target
```

[Service]

```
WorkingDirectory=/home/$USER/pinionai-streamlit-agent
EnvironmentFile=/home/$USER/pinionai-streamlit-agent/.env
ExecStart=/home/$USER/pinionai-streamlit-agent/.venv/bin/gunicorn -w 2 -k
uvicorn.workers.UvicornWorker app:app --bind 127.0.0.1:8000
Restart=always
RestartSec=5
User=$USER
Group=$USER
```

[Install]

```
WantedBy=multi-user.target
```

Enable and start it:

```
sudo systemctl daemon-reload
sudo systemctl enable pinionai-slack-marketplace
sudo systemctl start pinionai-slack-marketplace
sudo systemctl status pinionai-slack-marketplace
```

12. Validate before submission

Before submitting to the Slack Marketplace, confirm that:

- the public HTTPS domain is reachable
- Slack can verify the Request URL(s)
- public distribution is enabled
- your app profile and compliance fields are complete
- the app works in a real workspace

If you still see Slack submission errors, re-check:

- public distribution is turned on
- you are not relying on Socket Mode as the submission path
- your Request URLs are verified and reachable
- your app has a support URL and privacy policy

13. Update instructions

When you update the deployment later:

```
cd /home/$USER/pinionai-streamlit-agent
git pull
source .venv/bin/activate
```

```
pip install -r requirements.txt  
sudo systemctl restart pinionai-slack-marketplace
```

If you use Docker instead of a direct Python service, you can rebuild and restart the container:

```
docker build --no-cache -t pinionai-slack .  
docker run -p 3000:3000 pinionai-slack
```